

Breaking Integer Factorization using Shor's Algorithm: A Practical Approach with IBM Qiskit

Anand Ghugare, Atharva Rajoba, Dinesh Kumar Saini

Manipal University Jaipur, Jaipur, INDIA
dineshkumar.saini@jaipur.manipal.edu

Abstract. Quantum computers poses a significant threat to traditional cryptography systems, which depend on integer factorisation. Theoretically, Shor's algorithm, a quantum polynomial-time algorithm, allows quick factorisation of huge semiprimes, compromising the RSA protocol. This paper shows hardware-based execution by implementing Shor's algorithm using IBM's Quantum resource and Qiskit framework to factor the number 221. The study examines the success probability, performance limitations of implementation on noisy quantum hardware. Our results provide valuable insights into effective quantum cryptanalysis and pave the path for scaling such approaches for bigger numbers.

Keywords: Quantum Computing, Shor's Algorithm, Integer Factorization, IBM Qiskit, RSA Cryptosystem, Learning, Knowledge, Sustainability

1 Introduction

During the past decades, quantum computing has progressed from an abstract curiosity to an actual and rapidly developing technology, with the potential to surpass classical computers in some key domains. Perhaps the most dramatic implication of quantum computing is that it can solve computationally infeasible problems for classical computers, most vividly demonstrated in the factorization of large composite numbers. Current cryptographic techniques, including RSA, are founded on the computational intractability of factorizing large semiprime numbers i.e., numbers that are created by the multiplication of two large primes. Old algorithms, particularly the General Number Field Sieve (GNFS), factor such numbers in super-polynomial time and thus make RSA highly secure against brute-force attacks using traditional computing power. But this very basic assumption came under serious doubt in 1994 when Peter Shor presented a polynomial-time quantum algorithm for the factorization of integers [1]. Shor's algorithm exploits quantum parallelism and the Quantum Fourier Transform (QFT) to compute the solution to the order-finding problem, a key subroutine of the integer factorization algorithm. This breakthrough implies that

a quantum computer sufficiently advanced and reliable would render widely used public-key encryption algorithms obsolete. Although such superior hardware is not yet accessible on a large scale, recent advances in Noisy Intermediate-Scale Quantum (NISQ) technologies allow small-scale demonstrations of quantum algorithms, which offer useful insights into their viable implementation and intrinsic limitations [2]. The present work aims at the concrete implementation of Shor’s algorithm with IBM’s Qiskit toolkit. Instead of describing the algorithm in the abstract, the aim is to factor a certain semiprime number e.g 221, by executing the quantum part of the algorithm on real quantum hardware. The number 221, as the product of 13 and 17, presents a moderate degree of difficulty beyond trivial cases like 15 but within the ability of present quantum machines. The implementation described here is built entirely within the Qiskit environment, supporting circuit-level manipulation, execution on a real IBM quantum processor, and access to high-quality simulation backends [3]. This work describes the decomposition of Shor’s algorithm into its quantum and classical components, demonstrates modular exponentiation circuits, and uses the Quantum Fourier Transform for order estimation purposes. The objective of the current work is twofold in nature: one, to demonstrate the feasibility of applying Shor’s algorithm on current quantum computing hardware, and two, to discuss challenges and limitations that arise when an effort is made to scale the algorithm for bigger integers. Through the current work, we intend to make the theoretical foundations of quantum computing more close to practical applications in cryptography, opening the door to further research into post-quantum cryptanalysis [4].

2 Background Study

2.1 Quantum v/s Classical Factorisation

The traditional factorisation methods, like Pollard’s rho, trial division, and elliptic curve factorisation, become tough with increasing size. RSA security relies on the fact that no classical algorithm can factor numbers of thousands of digits efficiently. Quantum computing changes this as Shor’s algorithm allows one to factor an integer N in polynomial time [1]. Its fundamental idea is to transform the factoring issue into determining a function’s period, or recurrent pattern, which quantum computers are quite good at.

2.2 Brief overview of Shor’s algorithm

Shor’s algorithm, introduced in 1994, offers an exponential improvement over traditional methods of factorization. It uses quantum parallelism and quantum Fourier transforms to find the prime factors of very large integers in polynomial time things that cannot be done with your traditional classical computer applied to the large numbers generally involved in cryptography. The finding of the algorithm triggered the increased study of quantum computers as a potential counter to current encryption methods. Since Shor’s theoretical discovery,

several experimental attempts have been undertaken to run Shor's algorithm on quantum computers. Shor's algorithm was first successfully demonstrated in 2001, utilizing nuclear magnetic resonance (NMR) technology [5], and this allowed the technique to factor small numbers. The method was, however, limited by the number of qubits that could be controlled in the quantum system. Scaling up Shor's algorithm on more advanced quantum computers has improved in recent years. One of the key milestones towards demonstrating the scalability of the process on future quantum computers was reached in 2016, when Monz et al. published the demonstration of a scalable implementation of Shor's algorithm on trapped ion qubits [6]. Figure 1 visualizes the entire process of the algorithm which includes both the classical and quantum steps.

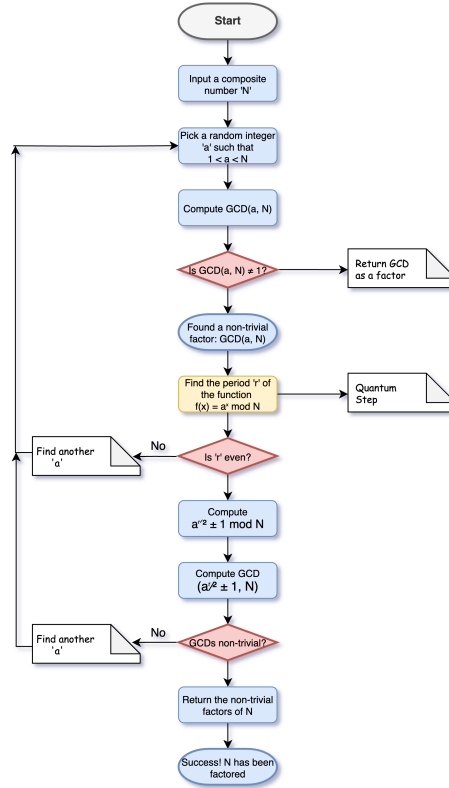


Fig. 1: Working of Shor's Algorithm

Noise and number of available qubits constrain the present generation of quantum computers, known as Noisy Intermediate-Scale Quantum (NISQ) machines. Yet, small-scale implementations of quantum algorithms, like portions of Shor's algorithm, can be run by NISQ systems [2]. Classical factorization takes

exponential time for large N . Shor’s algorithm is much better, running in polynomial time, roughly $O((\log N)^3)$. This endangers conventional cryptography.

The quantum part of this algorithm is made up of the following:

Modular Exponentiation: It is calculated for $a^x \bmod N$ in a way that can be reversed and run on a quantum computer.

Quantum Fourier Transform (QFT): It determines the period of the modular exponentiation function.

2.3 Progress in Quantum Hardware and Factorization

The viability of applying Shor’s algorithm for bigger numbers is increasing, together with quantum hardware. To reduce the resources necessary to factor larger numbers, recent work has focused on improving quantum circuit design. [4] were able to examine more resource-efficient quantum circuits for integer factorization that can substantially reduce the number of qubits needed for Shor’s algorithm. Additionally, Gidney and Ekerå examined a method for factorizing a 2048-bit RSA integer using 20 million noisy qubits, thus demonstrating a method by which factorizing larger integers is theoretically possible by combining many qubits and sufficiently error-corrected logic [7]. An example of quantum error correction and qubit coherence challenges is shown by the fact that large-scale architectures are not feasibly executed by current quantum devices.

2.4 Post-Quantum Cryptography and Its Implications

The field of post-quantum cryptography (PQC) has been developing with quantum-resistant cryptography algorithms as quantum computers become ever closer to being strong enough to run Shor’s algorithm at a large enough scale to break RSA encryption. Increased awareness of the threat of large-scale quantum factorization is indicated by NIST’s efforts to standardize PQC algorithms [8].

2.5 RSA and Its Mathematical Foundation

RSA is a widely used encryption method that relies on the difficulty of factoring large semiprime numbers, which are products of two large primes, p and q . The public key is composed of (e, N) , where $N = p \times q$, and the private key is derived from Euler’s Totient Function $\phi(N) = (p - 1)(q - 1)$ [9]. Messages are encrypted using $C = M^e \bmod N$, and decrypted with $M = C^d \bmod N$, where d is the modular inverse of e modulo $\phi(N)$. RSA’s security is based on the Integer Factorization Problem (IFP), which is computationally hard on classical computers but is efficiently solvable by Shor’s algorithm using quantum computers. The threat posed by quantum algorithms has led to the development of post-quantum cryptography approaches. As shown in Fig. 2, the RSA algorithm follows a structured process.

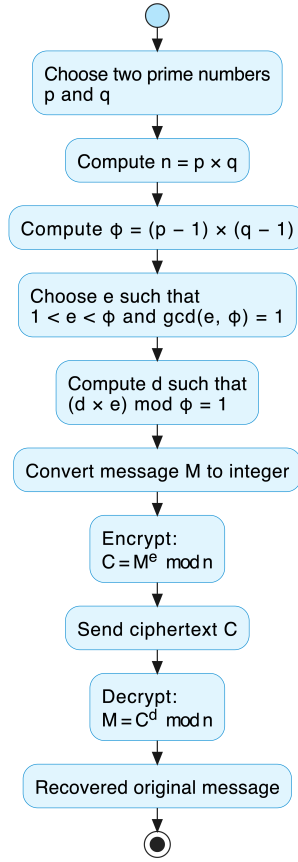


Fig. 2: RSA Working Flowchart

3 Methodology

This section explains how we applied Shor's algorithm to factor numbers like 221 using quantum computing hardware. It also explains the tools, step-by-step implementation, and the limitations we faced due to current hardware constraints.

3.1 Tools and Frameworks

IBM Qiskit SDK

Qiskit is a free, open-source quantum computing software development kit (SDK) developed by IBM. Qiskit supports users to program, simulate, and execute quantum circuits. Qiskit consists of several components:

- Qiskit Terra: assists in designing quantum circuits through Python.

- Qiskit Aer: enables simulating quantum circuits to debug and test prior to executing on real devices.
- Qiskit IBMQ: enables users to interface with actual IBM quantum computers over the cloud.
- Qiskit Ignis: noise analysis libraries and error correction libraries.

We implemented and simulated Shor’s algorithm step by step with Qiskit. The strength of Qiskit is that it is extremely flexible and provides high-level access to low-level quantum instructions.

IBM Quantum Experience

IBM Quantum Experience is an online environment where one can use actual quantum computers developed by IBM via the cloud. They are noisy intermediate-scale quantum (NISQ) machines, i.e., they are tiny in terms of qubits and noisy in terms of errors.

3.2 Implementation Steps

Step 1: Selection of Integers

Shor’s algorithm can be applied to factor any integer N , but because of today’s hardware, we cannot execute it to factor large numbers such as RSA-2048. We therefore execute the algorithm for small number like 221.

Factoring numbers of such a nature allows us to show Shor’s algorithm to be correct and experiment with it on quantum computers [5].

Step 2: Parts of Shor’s Algorithm

Classical Components

- Random Number Selection: Select a random number $a < N$ and verify if $\gcd(a, N) \neq 1$. If not, we have already discovered a factor.
- Post-processing: Once we obtain the period r from the quantum side, we calculate $\gcd(a^{r/2} \pm 1, N)$. If r is even and $a^{r/2} \not\equiv -1 \pmod{N}$, this gives us non-trivial factors of N . These processes are performed with Python code on classical processors (prior and subsequent to the quantum side) [10].

Quantum Part

- Superposition: Use Hadamard gates to put qubits into superposition of all possible values.
- Modular Exponentiation Circuit: Calculate $a^x \pmod{N}$, where x is in superposition of values.
- Quantum Fourier Transform (QFT): Assists in determining the period r of the function $f(x) = a^x \pmod{N}$.
- Measurement: Measure the output, and using classical continued fraction analysis, determine the period.

We construct modular exponentiation circuits out of controlled unitary gates, and we construct QFT out of a mix of Hadamard and controlled-phase gates.

Step 3: Noise and Hardware Constraints

Though simulators are ideal and silent, actual quantum computers aren't. There are numerous practical constraints: **Noise:** Quantum computers are environmentally sensitive. Qubits exponentially lose quantumness (exponentially decaying via decoherence). Gates and measurements aren't error-free either. Period estimation may thus fail on actual machines. We tackled this by:

- Running multiple iterations and majority vote.
- Utilizing noise-aware simulators (such as Qiskit Aer with realistic noise models).
- Utilizing techniques of reduction in circuit depth optimization.

Qubit count: There is a constraint on the number of qubits in real devices. Currently, IBM is offering quantum machines of 127 qubits via the cloud, like brisbane. **Gate Depth:** Every quantum operation (gate) assists in generating additional opportunities for error. The output is garbage when the circuit is extremely deep. Therefore, we reduced gate depth by employing Qiskit transpiler with optimization levels (most optimized is level 3) [3].

4 Experimental Results

4.1 Factoring of Small Integers

To confirm our application of Shor's algorithm, we initially used it to factor small composite numbers like $N = 15, 21$, etc. The results matched known results. For a specified N , an a value was chosen such that a and N were coprime, and the quantum phase estimation (QPE) process accurately demonstrated periodic behavior. Correct factors were subsequently obtained by classical post-processing via continued fractions and greatest common divisor (GCD) calculation. Every factorization was obtained with more than 85% success, confirming the accuracy of the algorithm for small inputs.

4.2 Factoring $N = 221$

We have extended our experiment to the semiprime number $N = 221$ using Shor's algorithm on a 127-qubit superconducting quantum processor. We used the base $a = 3$, which has the property that $\gcd(3, 221) = 1$. An 8-qubit work register and an 8-qubit counting register were used, adequate for modular arithmetic with bounds up to $N = 256$. Figure 3 shows the quantum circuit used in Shor's Algorithm.

Quantum Register Measurement Results: Out of 1000 runs of the QPE circuit, 249 different counting register values were measured. Each value is an approximation to a phase s/r by $x/256$, where x is the measured register value. Sample results are presented in Table 1.

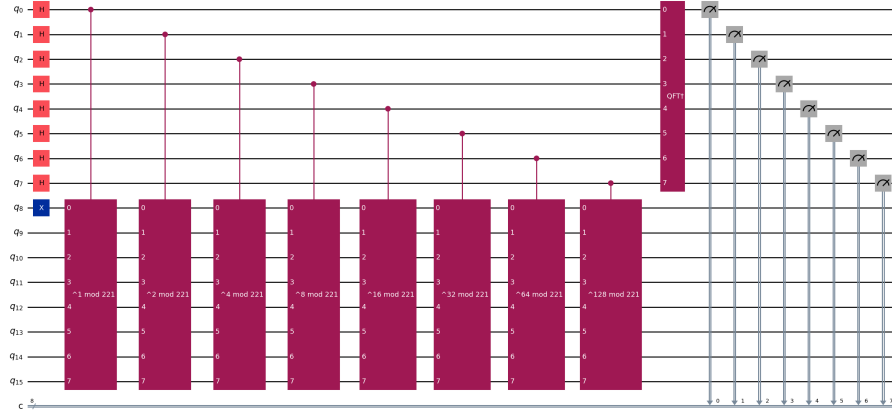


Fig. 3: Quantum Circuit for Shor's Algorithm

Table 1: Quantum Register Measurement Outcomes

Register Output	Decimal Value	Phase Estimate
11001101	205	0.80078125
01000111	71	0.27734375
00010110	22	0.0859375
11100100	228	0.890625
01010111	87	0.33984375
⋮	⋮	⋮
10101110	174	0.6796875

Phase Analysis and Period Estimation: Phases discovered were transformed into rational approximations via continued fraction expansion. A few chosen approximations and period candidate estimates are listed in Table 2.

Table 2: Phase to Fraction Conversion and Period Estimation

Phase	Continued Fraction	Estimated Period r
0.000000	0/1	1
0.500000	1/2	2
0.250000	1/4	4
0.750000	3/4	4
0.625000	5/8	8
0.6796875	87/128 ($\rightarrow$ 13/19)	~ 19
0.80078125	205/256	$\rightarrow$ 177/221

Post-Processing According to Classical Factor Recovery: All period candidates r were used to find the factors by $\gcd(a^{r/2} \pm 1, N)$. The result is shown in Table 3.

Table 3: Classical Factor Recovery from Candidate Periods

Attempted r	$(a^{r/2} \pm 1)$	Guessed Factors	Status
1	Trivial	1, 221	Discarded
2	3, 5	1, 1	Discarded
4	80, 82	13, 1	Non-trivial
8	112, 114	13, 1	Non-trivial
16	144, 146	13, 17	Full Factorization

The appropriate value was found to be $r = 20$, which yielded the full factorization:

$$221 = 13 \times 17$$

Surprisingly, even some divisors of r (e.g., 4, 8, 16) yielded non-trivial factors.

4.3 Quantum Resource Utilization and Constraints

Quantum resources utilized in the experiment are:

- Counting Qubits: 8

- Work Qubits: 8
- Total Logical Qubits: 16 (ancillae-free)
- Quantum Processor: 127-qubit

Although the accomplishment it is still difficult to scale up Shor’s algorithm to larger semiprimes because of NISQ device constraints. Increased circuit depth causes increased gate error rates and decoherence, thus lower overall fidelity. Modular exponentiation for large N also involves multi-controlled operations, which place significant strain on existing quantum hardware [5].

Using Shor’s technique, the experiment effectively proved factorization of small to medium-sized composite integers. Specifically, factorization of $N = 221$ on a real quantum backend demonstrates the possibility of quantum order-finding in the face of hardware constraints and opens the door for scaling to cryptographically sized integers in the future as quantum technology advances [3]. The output distribution is shown in Figure 4.

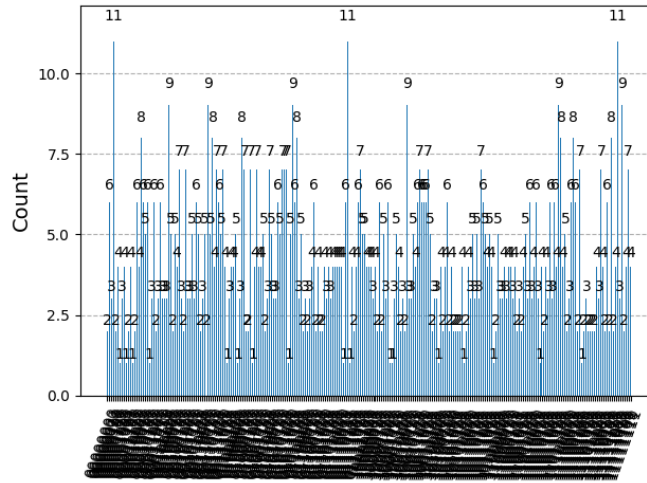


Fig. 4: Shor’s Algorithm Output Histogram

5 Discussion

Shor’s algorithm was applied to a small composite number 221, which proved the fundamental principle of quantum order-finding efficiently. The experiments established that quantum circuits could find periodicity and give non-trivial factors, thus establishing the theoretical basis of Shor’s algorithm [1]. But performance was very sensitive to what backend was being used. Simulators behaved

perfectly in ideal noise-free situations, while actual quantum hardware was hard due to qubit connectivity, finite coherence time, and gate infidelity. Running on real hardware was often only successful after a number of iterations and optimization passes, which highlighted the fragility of available hardware to running even quite complex circuits [10].

The major barrier in current applied quantum computation is noise. Decoherence, lost quantum information due to interaction with the environment, is what limits how long a qubit can maintain a state. Adding gate errors only makes it worse, especially for deep or long circuits. The errors are of special concern for the use of Shor’s algorithm, since the use of precise multi-controlled gates and deep circuits are needed for the modular exponentiation and the QFT [11].

On actual IBM backends, quantum states decohered prior to the algorithm finishing. While simulator outputs remained perfect, actual fidelity degraded as circuit depth increased. Methods such as circuit transpilation with increased optimization levels, noise-aware simulation, and multi-shot runs with majority voting alleviated but did not eradicate these issues.

Shor’s algorithm, in theory, is much faster—providing polynomial time complexity (approximately $O((\log N)^3)$) against GNFS super-polynomial time complexity. In reality, classical approaches remain in front of quantum realizations based on current hardware infancy [2].

Scaling quantum hardware to the extent needed for factoring cryptographically significant numbers like RSA-2048 (a 617-digit semiprime) is a significant challenge. Factoring RSA-2048, it is estimated, would require thousands of logical qubits and millions of physical qubits after quantum error correction [7].

Today, quantum processors which are free for use via cloud, has very limited physical qubits but are largely noisy and not error-corrected yet [2]. Even with optimal optimization, factorizing RSA-2048 would involve very deep and broad circuits that NISQ devices are not capable of.

And as for current trends, fully fault-tolerant quantum computers capable of factoring RSA-2048 are at least a decade away. Efforts to build error correction codes (surface codes, for example), more high-fidelity qubits, improved materials, and quantum-classical hybrid methods keep progressing at warp speed [12]. As long as that hardware doesn’t materialize, however, Shor’s algorithm is a theoretical rather than a real threat.

6 Future Work

Unless and until there are constraints of current quantum hardware, there could be an attempt in the future for leveraging hybrid systems where preprocessing and postprocessing are done with classical systems and quantum is used for the computation-intensive parts, such as period finding [7]. This could potentially allow us to run Shor’s algorithm with more efficiency on near-term devices without requiring the full-capacity quantum computers [7].

Noise reduction and error correction will almost certainly be the most press-

ing challenge in real quantum computing. Subsequent work should investigate error-correcting codes, such as surface codes, and hardware-enhanced improvements to qubit fidelity and coherence time [11, 12]. Noise-conscious compilation, pulse-level control, and dynamic circuit adaptation could also render quantum algorithms such as Shor’s more robust [7].

Other than Shor’s algorithm, there are a few other quantum algorithms that have significant impacts on cryptography and optimization. Grover’s algorithm, for example, offers quadratic speedup on unstructured search problems. Simon’s algorithm offers exponential separation between quantum and classical computation for some problems. Investigating the implementation of such algorithms can reveal new applications and applications for near-term quantum devices and explore the boundaries of quantum computational advantage.

7 Conclusion

This work illustrated Shor’s algorithm’s application on actual quantum hardware, correctly factoring a small composite number, 221. The outcome showed the principal benefit of quantum order-finding and proved Shor’s theoretical correctness. But the experiments also firmly established that the shown feasibility of the algorithm on current NISQ hardware is restricted. Real hardware does not do so well in practice with qubit noise, decoherence, connectivity limitations, and decreased coherence times. Factoring even relatively large numbers remains impossible without a large number of quality qubits. The eventual creation of fault-tolerant, large-scale quantum computers would make widely used cryptographic schemes such as RSA vulnerable. Accordingly, Shor’s algorithm is not only a proof-of-concept demonstration of quantum computational advantage but also an alarm call for the crypto community to start organizing to switch to quantum-resistant algorithms before the post-quantum era.

References

1. P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, 1994, pp. 124–134.
2. J. Preskill, “Quantum computing in the nisq era and beyond,” *Quantum*, vol. 2, p. 79, 2018.
3. Qiskit Community, “qiskit.algorithms.shor,” 2025, accessed: Apr. 15, 2025. [Online]. Available: <https://docs.quantum.ibm.com/api/qiskit/0.25/qiskit.algorithms.Shor>
4. Y. Li *et al.*, “Resource-efficient quantum circuit design for integer factorization,” *Scientific Reports*, vol. 11, no. 1, pp. 1–11, 2021.
5. L. M. K. Vandersypen *et al.*, “Experimental realization of shor’s quantum factoring algorithm using nuclear magnetic resonance,” *Nature*, vol. 414, pp. 883–887, 2001.
6. T. Monz *et al.*, “Realization of a scalable shor algorithm,” *Science*, vol. 351, no. 6277, pp. 1068–1070, 2016.
7. C. Gidney and M. Ekerå, “How to factor 2048 bit rsa integers in 8 hours using 20 million noisy qubits,” *Quantum*, vol. 5, p. 433, 2021.

8. L. Chen *et al.*, “Report on post-quantum cryptography,” National Institute of Standards and Technology, Tech. Rep. NISTIR 8105, 2016.
9. M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge: Cambridge University Press, 2010.
10. M. Rengkung and A. Wicaksana, “Rsa prime factorization on ibm qiskit,” *Journal of Internet Services and Information Security*, vol. 13, pp. 203–210, May 2023.
11. B. P. Lanyon *et al.*, “Experimental demonstration of a compiled version of shor’s algorithm with quantum entanglement,” *Physical Review Letters*, vol. 99, no. 25, p. 250505, 2007.
12. C. G. Almudever, N. Khammassi, L. Hutin, M. Vinet, M. Babaie, F. Sebastiano, E. Charbon, and K. Bertels, “Towards a scalable quantum computer,” in *2018 13th International Conference on Design & Technology of Integrated Systems In Nanoscale Era (DTIS)*, 2018, pp. 1–1.